

```

#include <iostream>
#include <string>
#include <queue>
#include <unordered_map>
#include <Windows.h>
#include <map>

using namespace std;

struct Node {
    char ch;
    int freq;
    Node* left, * right;
};

Node* getNode(char ch, int freq, Node* left, Node* right) {
    Node* node = new Node();

    node->ch = ch;
    node->freq = freq;
    node->left = left;
    node->right = right;

    return node;
}

struct comp {
    bool operator()(Node* l, Node* r) {
        return l->freq > r->freq;
    }
};

void encode(Node* root, string str, unordered_map<char, string>& huffmanCode) {
    if (root == nullptr)
        return;

    if (!root->left && !root->right) {
        huffmanCode[root->ch] = str;
    }

    encode(root->left, str + "0", huffmanCode);
    encode(root->right, str + "1", huffmanCode);
}

void decode(Node* root, int& index, string str) {
    if (root == nullptr) {
        return;
    }

    if (!root->left && !root->right) {
        cout << root->ch;
        return;
    }

    index++;

    if (str[index] == '0')
        decode(root->left, index, str);
    else
        decode(root->right, index, str);
}

```

```

unordered_map<char, string> buildHuffmanTree(string text) {
    unordered_map<char, int> freq;
    for (char ch : text) {
        freq[ch]++;
    }

    priority_queue<Node*, vector<Node*>, comp> pq;

    for (auto pair : freq) {
        pq.push(getNode(pair.first, pair.second, nullptr, nullptr));
    }

    while (pq.size() != 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();

        int sum = left->freq + right->freq;
        pq.push(getNode('\0', sum, left, right));
    }

    Node* root = pq.top();

    unordered_map<char, string> huffmanCode;
    encode(root, "", huffmanCode);

    int count = 0;
    cout << "Huffman Codes are :\n" << '\n';
    for (auto pair : huffmanCode) {
        cout << pair.first << " " << pair.second << '\n';
        count++;
    }

    cout << "\nOriginal string was :\n" << text << '\n';

    string str = "";
    for (char ch : text) {
        str += huffmanCode[ch];
    }

    cout << "\nEncoded string is :\n" << str << '\n';

    int index = -1;
    cout << "\nDecoded string is: \n";
    while (index < (int)str.size() - 2) {
        decode(root, index, str);
    }

    return huffmanCode;
}

unordered_map<char, string> helpFunction(string text) {
    unordered_map<char, int> freq;
    for (char ch : text) {
        freq[ch]++;
    }

    priority_queue<Node*, vector<Node*>, comp> pq;

    for (auto pair : freq) {
        pq.push(getNode(pair.first, pair.second, nullptr, nullptr));
    }

```

```

    }

    while (pq.size() != 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();

        int sum = left->freq + right->freq;
        pq.push(getNode('\0', sum, left, right));
    }

    Node* root = pq.top();

    unordered_map<char, string> huffmanCode;
    encode(root, "", huffmanCode);

    return huffmanCode;
}

map<char, int> numberEachSymbol(string text) {
    map<char, int> check;
    auto jt = check.begin();
    auto k = check.end();
    for (int i = 0; i < text.size(); i++) {
        bool flag = 0;
        for (auto it = check.begin(); it != k; it++) {
            if (it->first == text[i]) {
                it->second++;
                flag = 1;
            }
        }
        if (flag == 0) {
            check.insert(pair<char, int>(text[i], 1));
        }
    }

    return check;
}

float compressionRatio(string text, map<char, int> check, int characters, int
bitCharacter, int n) {
    unordered_map<char, string> help = helpFunction(text);
    float sum = 0.0;
    for (auto it = check.begin(); it != check.end(); it++) {
        sum += it->second * help[it->first].size();
    }

    float result = (characters * bitCharacter) / (sum);

    return result;
}

unordered_map<char, float> probability(map<char, int> symbol, float n) {
    auto k = symbol.end();
    unordered_map<char, float> res;
    for (auto it = symbol.begin(); it != k; it++) {
        res.insert(make_pair(it->first, it->second / n));
    }

    return res;
}

```

```

float dispersion(map <char, int> check, string text, float n) {
    unordered_map<char, string> help = helpFunction(text);
    float Lsr = 0.0;
    int i = 0;
    auto k = check.end();
    unordered_map<char, float> probabilityOccurrence = probability(check, n);

    for (auto it = probabilityOccurrence.begin(); it != probabilityOccurrence.end();
it++)
        cout << (*it).first << ' ' << it->second << '\n';

    for (auto it = check.begin(); it != k; it++) {
        Lsr += help[it->first].size() * probabilityOccurrence[it->first];
    }

    float res = 0.0;
    for (auto it = check.begin(); it != k; ++it) {
        res += probabilityOccurrence[it->first] * ((help[it->first].size() - Lsr)
* (help[it->first].size() - Lsr));
    }

    return res;
}

int main() {
    setlocale(LC_ALL, "Russian");
    string text = "Код Хаффмана";

    map <char, int> check;
    check = numberEachSymbol(text);

    auto it = check.begin();
    for (; it != check.end(); it++)
        cout << (*it).first << ' ' << it->second << '\n';

    buildHuffmanTree(text);

    cout << '\n';

    cout << "Коэффициент сжатия: ";
    float res = compressionRatio(text, check, text.size(), 5, check.size());

    cout << res << '\n';

    cout << "Дисперсия: ";

    float disp = dispersion(check, text, text.size());

    cout << disp << '\n';

    return 0;
}

```

Результат работа программы:

```
Консоль отладки Microsoft Visual Studio
К 1
Х 1
а 3
д 1
м 1
н 1
о 1
ф 2
1
Huffman Codes are :
К 000
д 1101
ф 101
м 001
а 01
1 1110
Х 100
о 1100
н 1111

Original string was :
Код Хаффмана

Encoded string is :
0001100110111101000110110100101111101

Decoded string is:
Код Хаффмана
Коэффициент сжатия: 1.62162
Дисперсия: 0.576389
```